1. Кои член-функции не могат да бъдат обявени за виртуални? Обяснете защо

2. Какво представлява overriding на функция в C++?
   Дайте примери.

3. Обяснете каква е разликата между виртуални и чисто виртуални функции. Обяснете особеностите, свързани с наличието на чисто виртуални функции за даден клас и за неговите наследници.

4. Изберете подходящи спецификатори за достъп и атрибут за област в йерархията base <- der:

```
class base                class der : ........ base
{..............: int a1;  {............: int d1;
 ..............: int a2;   ............: int d2();
 ..............: int a3();  ............: int d3();
};                        };
```

Така че фрагментът:

```
int der::d3(){
       std::cout << a1 << "\n";
       std::cout << d1 << " " << d2 << "\n";
       return a1 + a3();
};
...
der x, y;
x.a2 = 25;
x.d3();
```

да не предизвиква синтактични грешки, а всеки следващ ред да предизвиква грешка по време на компилация:

```
y.a1;
y.d1 = 22;
x.d2 = 15;
y.a3();
```

5. За шаблона на функцията

```
template <typename T, typename U, typename V>
T func(U& u, V v) {
  T res = u;
  res += v;
  return res;
}
```

Какви ограничения се налагат от шаблона на функцията за типовете T, U и V? Направете едно примерно обръщение към функция, която се генерира по шаблона.

6. Намерете резултата от изпълнението на програмата. Напишете точно какво ще се изведе на стандартния изход. Посочете, ако има някакви грешки по време на компилация или изпълнение. Обосновете отговора си.

```
#include <iostream>
class Base {
public:
  Base(int a = 15): x(a) {}
  void showX() {
    std::cout << "x = " << x << "; ";
  }
  virtual void modifyX() {
    x *= 3;
  }
protected:
  int x;
};

class Derived : public Base {
public:
  void modifyX() {
    x /= 3;
  }
};

int main(){
  Base b;
  Derived d;
  b.showX();
  d.showX();
  Base* pB;
  pB = &b; pB->modifyX(); pB->showX();
  pB = &d; pB->modifyX(); pB->showX();
  return 0;
}
```

7. Огледайте следния код и открийте грешките в него. За всяка обяснете защо е грешка и защо това е правилната корекция

```
template<typename T>
class Base1 {
  Base1(char* s1, char* s2) : m_s1(m_s1) {
    try {
      m_s2 = new char[strlen(s2) + 1];
      strcpy(m_s2, s2);
    }
    catch (...) {
      delete[] m_s1;
      delete[] m_s2;
    }
  }
  Base1(Base1 other) : Base1(other.m_s1, other.m_s2) { }
  Base1& operator=(const Base1& other) {
    if (*this != other) {
      Base1 temp = other;
      std::swap(this, &temp);
    }
    return *this;
  }
```

```
  ~Base1() {
     delete[] m_s1;
     delete[] m_s2;
  }
  void setToDefault() {
    strcpy(m_s1, "");
    strcpy(m_s2, "");
  }
protected:
  virtual void display() {
    std::cout << m_s1 << '\t' << m_s2 << '\t' << std::endl;
  }
private:
  char* m_s1;
  char* m_s2;
};

template<typename T>
class Base2 {
public:
  Base2(int i) : m_i(i) { }
  virtual void display() {
    std::cout << m_i << std::endl;
  }
  void setToDefault() {
    std::cout << "No data to be changed!\n";
  }
  void setVal(T val) {
    m_val = std::move(val);
  }
public:
    const int m_i;
    T m_val;
};

class Derived : public Base1, Base2 {
public:
  Derived(const char* s1, const char* s2, int i) : Base2(i) { }
  void display() const {
    Base1::display();
    Base2::display();
  }
};

int main() {
  Derived* der = new Derived("a", "b", 1);
  Base1<int>* base1 = der;
  Base2* base2 = der;
  der->display();
  base1->display();
  base2->display();
  Derived other("A", "B", 2);
  other = *der;
  other.display();
  other.setToDefault();
  return 0;
}
```

8. Какво представляват статичните член-данни и функции?
   По какво се различават от обикновените?
   Как се инициализират статичните член-данни?
   До какво имат достъп статичните член-функции?